

Unconditional Branching and GOTO

GOTO Statement

Warning: Generally discouraged in modern programming (leads to "spaghetti code")

```
// C - Using goto (not recommended)
#include <stdio.h>
```

```
int main() {
    int i = 0;

    start: // Label
        if (i >= 5) {
            goto end; // Jump to end
        }
        printf("%d ", i);
        i++;
        goto start; // Jump back to start

    end:
        printf("\nDone!\n");
        return 0;
}
// Output: 0 1 2 3 4
```

Better Alternative - Structured Programming

```
// Better approach without goto
int main() {
    for (int i = 0; i < 5; i++) {
        printf("%d ", i);
    }
    printf("\nDone!\n");
    return 0;
}
```

Acceptable GOTO Use Cases

```
// Error handling in C (acceptable use)
int process_data() {
    FILE *file = fopen("data.txt", "r");
    if (file == NULL) goto error;

    int *buffer = malloc(1024);
    if (buffer == NULL) goto cleanup_file;

    // Process data...

    free(buffer);
cleanup_file:
    fclose(file);
    return 0;

error:
    return -1;
}
```

Return Statement (Structured Alternative)

```
def find_element(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i # Exit function immediately
    return -1 # Not found

numbers = [10, 20, 30, 40]
index = find_element(numbers, 30)
print(f"Found at index: {index}") # Output: 2
```

Guarded Commands: Non-Deterministic Selection

What is Non-Deterministic Selection?

Non-deterministic selection means that when multiple conditions (guards) are true at the same time, the program can choose any one of them to execute. The choice is not predetermined by the programmer.

This is different from traditional if-else statements where conditions are checked in a specific order and only the first true condition executes.

Dijkstra's Guarded Command Language (GCL)
Basic Syntax

```
if guard1 → statement1
[] guard2 → statement2
[] guard3 → statement3
fi
```

Key Difference: Deterministic vs Non-Deterministic Deterministic (Traditional if-else)

Python - Deterministic

```
x = 5
y = 5

if x >= y:
    print("x is greater or equal") # This ALWAYS executes first
elif y >= x:
    print("y is greater or equal") # Never reaches here when x == y

// **Result**: Always prints "x is greater or equal" when both are true
```

Non-Deterministic (Guarded Commands) // Pseudo-code GCL

```
if x >= y → print("x is greater or equal")
[] y >= x → print("y is greater or equal")
Fi
```

// **Result**: When x == y, either statement could execute (random choice)

Why Use Non-Deterministic Selection?

1. Parallel Programming

When you don't care which task executes first

2. Formal Verification

Easier to prove program correctness

3. Modeling Concurrent Systems

Represents real-world non-determinism

4. Fairness

No branch has priority over others

Example 1: Maximum of Two Numbers

Traditional Approach (Deterministic)

```
def max_deterministic(x, y):
    if x > y:
        return x
    elif y > x:
        return y
    else: # x == y
        return x # Always returns x when equal
```

Guarded Command Approach (Non-Deterministic)

```
import random

def max_nondeterministic(x, y):
    # Collect all true guards
    guards = []

    if x >= y:
        guards.append(('x >= y', x))
    if y >= x:
        guards.append(('y >= x', y))

    # If no guard is true, abort
    if not guards:
        raise ValueError("No guard is true (impossible for max)")

    # Randomly select one true guard
    chosen_guard, result = random.choice(guards)
    print(f"Chosen guard: {chosen_guard}")
    return result

# Test when x == y
print("Test 1: x=5, y=5")
for i in range(3):
    result = max_nondeterministic(5, 5)
    print(f" Result: {result}\n")

# Test when x > y
print("Test 2: x=10, y=3")
result = max_nondeterministic(10, 3)
print(f" Result: {result}")
...
```

****Output:****

...

Test 1: x=5, y=5
Chosen guard: x >= y

Result: 5

Chosen guard: $y \geq x$
Result: 5

Chosen guard: $x \geq y$
Result: 5

Test 2: $x=10, y=3$
Chosen guard: $x \geq y$
Result: 10
...

Notice: When both guards are true ($x=5, y=5$), different guards get chosen!